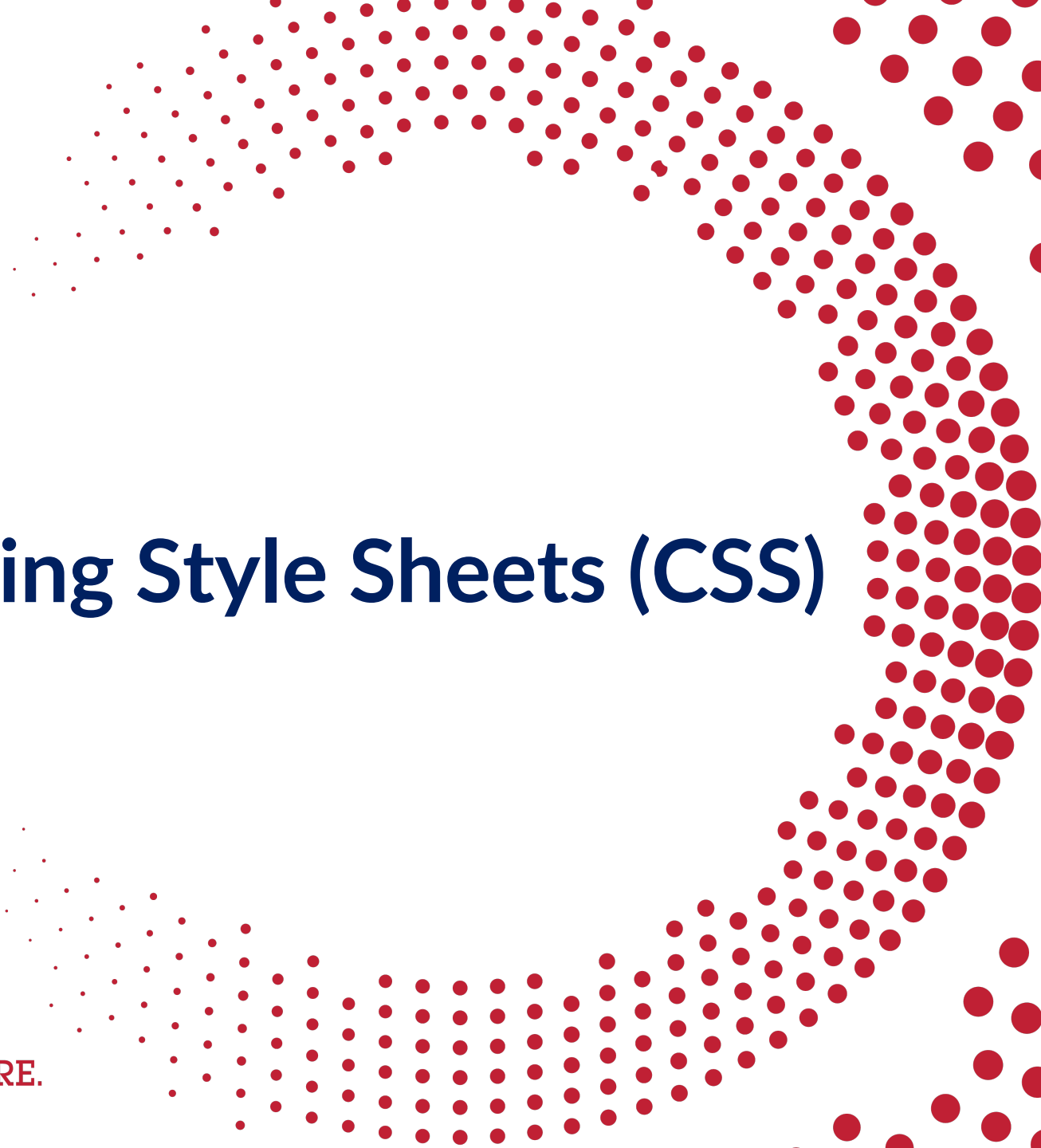




Cascading Style Sheets (CSS)

ONE LOVE. ONE FUTURE.

Motivation

Welcome to My Homepage

Use the menu to select different Stylesheets

Stylesheet 1

Stylesheet 2

Stylesheet 3

Stylesheet 4

No Stylesheet

Welcome to My Homepage

Use the menu to select different Stylesheets

- Stylesheet 1
- Stylesheet 2
- Stylesheet 3
- Stylesheet 4
- No Stylesheet

Same Page Different Stylesheets

This is a demonstration of how different stylesheets can change the layout of your HTML page. You can change the layout of this page by selecting different stylesheets in the menu, or by selecting one of the following links:
[Stylesheet1](#), [Stylesheet2](#), [Stylesheet3](#), [Stylesheet4](#).

No Styles

This page uses DIV elements to group different sections of the HTML page. Click here to see how the page looks like with no stylesheet:
[No Stylesheet](#).

https://www.w3schools.com/css/demo_default.htm

Content

1. Introduction
2. Responsive Design
3. CSS Framework
4. Webpage Layout



1. Introduction

1.1. Overview

1.2. Syntax

1.3. Selectors

1.4. Units and Important Properties

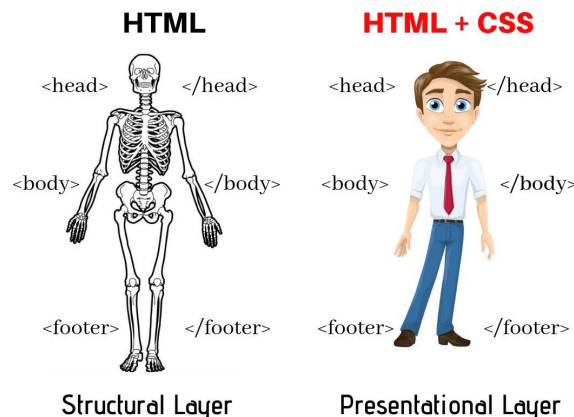
1.5. CSS Variables



What is CSS?

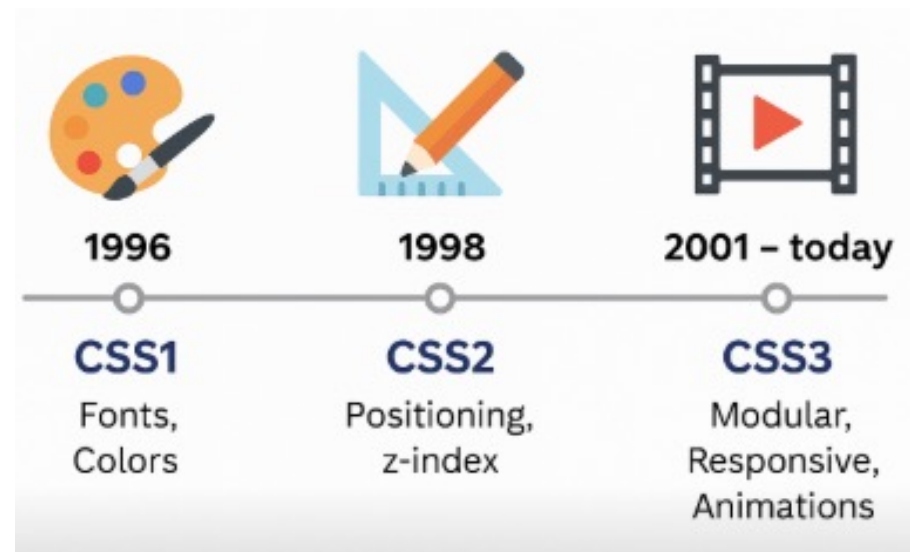
- CSS: Cascading Style Sheets
 - CSS is a style sheet language used to control the appearance of web pages
 - A **style sheet** contains rules that define colors, fonts, spacing, and layout, etc.
 - **Cascading** means multiple style rules can apply to the same element.
 - The browser chooses the rule with the highest priority.

HTML Vs CSS



History of CSS

- Håkon Wium Lie (1994). Idea of CSS
- CSS1 (1996)
 - Text formatting and colors
 - Properties: font-family, font-size, color, background
- CSS2 (1998)
 - Advanced layout
 - Properties: position, z-index, media types
- CSS3 (2001-now)
 - Modular structure
 - Responsive design support
 - Transition, animations



Advantages of CSS

- Separation of content & presentation
 - HTML handles content
 - CSS handles presentation
- Consistency across pages
 - Apply one presentation to multiple pages
- Easier maintenance
 - Update one CSS file → changes presentation everywhere
- Better SEO and device-friendliness
 - Clean HTML, faster loading
 - Responsive design for multiple device



1.2. Syntax

- Three ways to apply CSS

1. *Inline CSS*

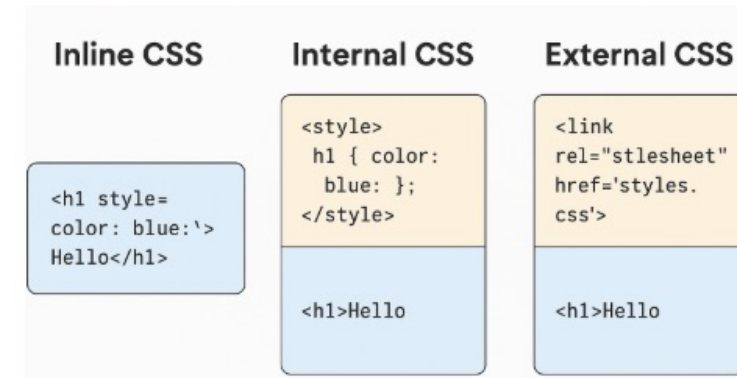
- CSS written directly inside an HTML tag
- Affects only to that HTML element

2. *Internal CSS*

- CSS defined inside <head>
- Affects only that HTML page

3. *External CSS*

- CSS Stored in a .css file and linked with <link>
- Can be reused across multiple pages



Inline CSS

- CSS rules are written in an element's style attribute
- Scope: affects that element only
- Syntax: `style="property1: value1; property2: value2;"`
- Features
 - Quick, apply to a single element
 - Hard to maintain

```
<!-- Inline CSS -->
```

```
<h2 style="color: red; font-family: Arial;">
```

```
  This is styled with Inline CSS
```

```
</h2>
```

This is styled with Inline CSS

Internal CSS

- Rules are written inside a `<style>` element within the `<head>` section of an HTML page
- Scope: affects that HTML page only
- Features:
 - Easier to manage than inline CSS
 - Cannot be reused across multiple pages

```
<head>
  <!-- Internal CSS -->
  <style>
    h2 {
      color: green;
      text-align: center;
    }
  </style>
</head>
```

This is styled with Internal CSS

External CSS

- Rules are written in a **separate .css file**
- Scope: can be reused across multiple pages
- Advantages:
 - Best practice for large projects
 - Promotes consistency across the whole site
 - Easier maintenance (update one file → changes everywhere)
 - Reduces HTML file size
- Limitations: Requires loading an additional file

CSS style.css > h2

```
1 h2 {  
2   color: darkblue;  
3   text-decoration: underline;  
4 }
```

```
<!-- External CSS -->  
<link rel="stylesheet" href="style.css">
```

Basic Syntax of Internal & External CSS

- Syntax

```
selector{  
    property: value;  
}
```

- Selector → chooses the element(s) to style (e.g., h1, #id)
- Property → the aspect you want to style (e.g., color, font-size)
- A group of declarations is enclosed in curly braces { }

```
h1 {  
    color: blue;  
    font-size: 24px;  
    text-align: center;  
}
```

Selector: h1 → all <h1> headings

Properties: color, font-size, text-align

Values: blue, 24px, center

1.3. Selectors

Selector	Example	Description
*	*	Selects all elements
element	p	Selects all <p> elements
.class	.intro	Selects all elements with class="intro"
#id	#firstname	Selects the element with id="firstname"

All elements

```
<!DOCTYPE html>
<html>
<head>
<style>
* {
  text-align: center;
  color: red;
}
</style>
</head>
<body>
<h1>Hello world!</h1>
<p>Me too!</p>
<p>And me!</p>
</body>
</html>
```

Hello world!

Me too!

And me!

Element, Class, ID

basic-selectors.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>Basic Selectors Example</title>
6    <style>
7      p {
8        color: red;
9      }
10     .intro {
11       color: blue;
12     }
13     #header {
14       background-color: lightgray;
15     }
16   </style>
17 </head>
18 <body>
19   <header id="header">
20     <h1>Welcome to CSS Selectors</h1>
21   </header>
22
23   <p>This paragraph is styled with the element selector.</p>
24
25   <p class="intro">This is a paragraph styled with a class selector.</p>
26 </body>
27 </html>
```

Welcome to CSS Selectors

This paragraph is styled with the element selector.

This is a paragraph styled with a class selector.

Descendant, Child, Sibling

- **Descendant A B**: selects all B that are anywhere inside A
- **Child A > B**: selects only B that are direct children of A
- **Adjacent sibling A + B**: selects the first B immediately after A (same parent).

Descendant Selector div p	<pre><div> <p>Paragraph 1<p> <section> <p>Paragraph 2</p> </section> </div></pre>
Child Selector ul > li	<pre> Top-level 1 Top-level 2 Nested </pre>
Adjacent Silbing Selector h1+p	<pre><h1>Article</h1> <p>Paragraph 1</p> <p>Paragraph 2</p></pre>

Example

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <style>
5        /* Descendant */
6        div p {
7          color: ■blue;
8        }
9        /* Child (only direct LI under the outer UL) */
10       ul > li {
11         color: ■red;
12       }
13       /* Adjacent sibling (only the first P after H1) */
14       h1 + p {
15         color: ■green
16       }
17     </style>
18   </head>
```

```
19   <body>
20     <div>
21       <p>Paragraph 1</p>
22       <section>
23         <p>Paragraph 2</p>
24       </section>
25     </div>
26
27     <ul>
28       <li>Top-level 1</li>
29       <li>Top-level 2 </li>
30       <ol>
31         <li>Nested li (NOT matched)</li>
32       </ol>
33     </ul>
34
35     <h1>Article</h1>
36     <p>Paragraph 1</p>
37     <p>Paragraph 2</p>
38   </body>
39 </html>
```

Paragraph 1

Paragraph 2

- Top-level 1
- Top-level 2
- 1. Nested li (NOT matched)

Article

Paragraph 1

Paragraph 2

Attribute Selector

- Select elements by attributes/values

Selector	Description	Example
[attr]	Elements that have the attribute , regardless of its value.	img[alt]{...}
[attr="value"]	Attribute value is exactly equal to "value"	input[type="text"]
[attr^="value"]	Attribute value starts with "value"	a[href^="https"]
[attr\$="value"]	Attribute value ends with "value"	img[src\$=".png"]
[attr*="value"]	Attribute value contains "value" as substring	[class*="btn"] → .btn, .btn-primary, .btn primary
[attr~="value"]	Attribute value contains "value" as a whole word (separated by spaces)	[class~="btn"] → .btn, .btn primary ✗ : btn-primary

```

<style>
  /* [attr] – has attribute */
  img[alt] {
    border: 2px dashed gray;
  }

  /* [attr="value"] – exact match */
  input[type="text"] {
    border: 2px solid blue;
  }

  /* [attr^="value"] – starts with */
  a[href^="https"] {
    color: green;
  }

  /* [attr~="value"] – contains whole word */
  [class~="btn"] {
    border: 1px solid black;
    background-color: lightblue;
  }
</style>

```

```

<h2>[attr]</h2>



<h2>[attr="value"]</h2>
<form>
  <input type="text" placeholder="Text field">
  <input type="password" placeholder="Password field">
</form>

<h2>[attr^="value"]</h2>
<a href="https://example.com">Secure Link</a><br>
<a href="http://example.com">Non-secure Link</a>

<h2>[attr~="value"]</h2>
<button class="btn primary">Primary Button</button>
<button class="btn-secondary">Secondary Button</button>

```

[attr]



[attr="value"]

Text field

Password field

[attr^="value"]

[Secure Link](#)

[Non-secure Link](#)

[attr~="value"]

Primary Button

Secondary Button

Pseudo-classes

- **Pseudo-classes** select elements based on a **special state** or **position** in the document
- Syntax: *selector:pseudo-class { property: value; }*
- State-based
 - :hover → when mouse is over an element
 - :active → when element is being clicked
 - :focus → when input is focused
- Structure-based
 - :first-child → the first child of its parent
 - :nth-child(n) → selects the nth child

```
a:hover { color: red; }  
input:focus { border: 2px solid blue; }  
p:first-child { font-weight: bold; }
```

Pseudo-classes Example

```
<style>
  /* :hover */
  a:hover {
    color: ■ red;
  }

  /* :active */
  a:active {
    color: ■ orange;
  }

  /* :focus */
  input:focus {
    border: 2px solid ■ blue;
  }

  /* :first-child */
  p:first-child {
    color: ■ darkgreen;
  }

  /* :nth-child */
  li:nth-child(2) {
    background-color: □ #f0f0f0;
  }
</style>
```

```
<body>
  <h1>Pseudo-class Demonstration</h1>
  <p>Move the mouse</p>
  <a href="#">Hover or click me</a>

  <p>Click inside the input field:</p>
  <input type="text" placeholder="Focus me">

  <h2>:first-child</h2>
  <div>
    <p>This is the first paragraph</p>
    <p>This is the second paragraph</p>
  </div>

  <h2>:nth-child</h2>
  <ul>
    <li>Item 1</li>
    <li>Item 2 (highlighted)</li>
    <li>Item 3</li>
  </ul>
</body>
```

Move the mouse

Hover or click me

Click inside the input field:

:first-child

This is the first paragraph

This is the second paragraph

:nth-child

- Item 1
- Item 2 (highlighted)
- Item 3

Pseudo-elements

- A **pseudo-element** allows you to **style specific parts** of an element.
- Syntax:
`selector::pseudo-element { property: value; }`
- Common Pseudo-element
 - `::before` → insert content before element
 - `::after` → insert content after element
 - `::first-line` → style the first line of text
 - `::first-letter` → style the first letter of text

Pseudo-elements Example

```
<style>
  /* ::first-letter */
  p::first-letter {
    color: ■red;
  }

  /* ::first-line */
  p::first-line {
    color: ■green;
  }

  /* ::after */
  p::after {
    content: " ✓";
  }
</style>
```

```
<p>
  This is the first line<br>
  This is the second line
</p>
```

```
<p>
  This is a paragraph
</p>
```

This is the first line
This is the second line ✓
This is a paragraph ✓

Pseudo-class vs. Pseudo-elements

Concept	Purpose	Example
Pseudo-class	Select elements based on state or position	:hover, :focus, :first-child
Pseudo-element	Style a specific part of an element	::before, ::after, ::first-letter

a:hover { color: red; } /* pseudo-class */

p::first-letter { font-size: 200%; } /* pseudo-element */

CSS Specificity (Priority)

- An algorithm determines which style declaration is applied to an element

```
<html>
<head>
  <style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>

<p id="demo" class="test" style="color: pink;">Hello World!</p>

</body>
</html>
```

```
<html>
<head>
  <style>
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>

<p class="test">Hello World!</p>

</body>
</html>
```

```
<html>
<head>
  <style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>

<p id="demo" class="test">Hello World!</p>

</body>
</html>
```

Specificity Hierarchy

Selector	Example	Description
Inline styles	<code><h1 style="color: pink;"></code>	Highest priority
Id selectors	<code>#navbar</code>	Second highest priority
Classes, attribute selectors and pseudo-classes	<code>.test, [type="text"], :hover</code>	Third highest priority
Elements and pseudo-elements	<code>h1, ::before, ::after</code>	Low priority
Universal selector and <code>:where()</code>	<code>*, where()</code>	No priority

!important keyword: overrides ALL styling rules

```
selector {  
    property: value !important;  
}
```

Example

```
<style>
#demo {color: blue;}
.test {color: green;}
p {color: red;}
</style>
</head>
<body>
<p id="demo" class="test" style="color: pink;">
Hello World!
</p>
```

Hello World!

```
<style>
p {
  background-color: yellow !important;
}
#myid {
  background-color: blue;
}
.myclass {
  background-color: gray;
}
</style>
</head>
<body>

<p style="background-color:orange;">This is a paragraph.</p>
<p class="myclass">This is a paragraph.</p>
<p id="myid">This is a paragraph.</p>
```

This is a paragraph.

This is a paragraph.

This is a paragraph.

1.4. Units & Important Properties

Absolute Units

- px (pixels)
 - Fixed relative to the **screen's resolution**.
 - Most commonly used in web design.
- pt (points)
 - 1 pt = 1/72 of an inch.
 - More common in print rather than web.
- cm (centimeters)
 - Physical measurement unit.
 - Rarely used on screens due to device variability.
- Features
 - Always fixed
 - Good for print, not ideal for responsive web

1.4. Units & Important Properties

Relative Units

- % (percentage)
 - Relative to the **parent element**.
 - Common for width, height, margin, padding.
- em
 - Relative to the **font-size** of the **parent element**.
 - Example: if parent = 16px, then 2em = 32px.
- rem (root em)
 - Relative to the **font-size** of the **root element (html)**.
- vw / vh (viewport width / height)
 - 1vw = 1% of viewport's width.
 - 1vh = 1% of viewport's height.
 - Useful for full-screen layouts and responsive design.

Units Example

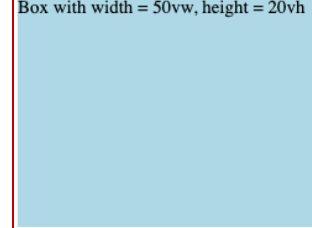
```
<style>
  html {
    font-size: 15px;
  }
  .rem-box {
    background: lightpink;
    font-size: 2rem;
  }
  .viewport-box {
    background: lightblue;
    width: 50vw; /* 50% of viewport width */
    height: 20vh; /* 20% of viewport height */
  }
</style>
```

```
<body>
  <p class="rem-box">
    Font-size: 2rem (relative to root)
  </p>

  <p class="viewport-box">
    Box with width = 50vw, height = 20vh
  </p>
</body>
```

Font-size: 2rem (relative to root)

Box with width = 50vw, height = 20vh

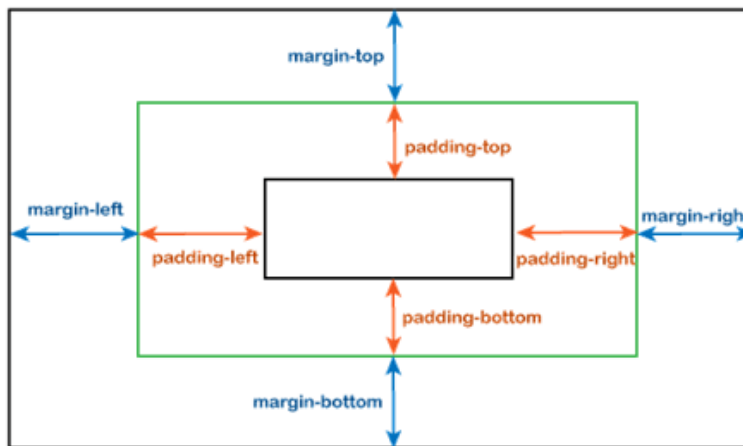


Color & Background

- Text color
 - `color: red;` → sets text color.
 - Supports: name (`red`), HEX (`#ff0000`), RGB (`rgb(255,0,0)`), HSL (`hsl(0,100%,50%)`).
- Background color
 - `background-color: #ff0000;`
 - Sets solid background behind an element.
- Background image
 - `background-image: url("bg.png");`
 - Allows adding image backgrounds.

Box Model

- Every element in CSS is a rectangular box.
- Box model consists of 4 layers:
 - Content → the actual text, image, or element inside
 - Padding → space between content and border
 - Border → surrounds the padding and content
 - Margin → outermost space, separates element from neighbors

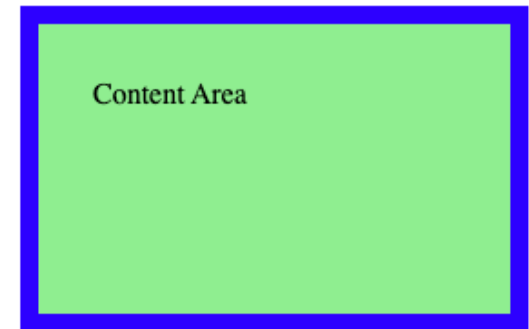


```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Box Model Demo</title>
  <style>
    .box {
      width: 200px;
      height: 100px;
      background-color: lightgreen; /* Content */
      padding: 30px;               /* Padding */
      border: 10px solid blue;     /* Border */
      margin: 50px;               /* Margin */
    }
  </style>
</head>
<body>
  <h1>CSS Box Model Demo</h1>
  <div class="box">
    Content Area
  </div>
  <p>
    The green part is Content,<br>
    the space inside border is Padding,<br>
    the blue outline is Border,<br>
    and the outside space is Margin.
  </p>
</body>
</html>

```

CSS Box Model Demo



The green part is **Content**,
the space inside border is **Padding**,
the blue outline is **Border**,
and the outside space is **Margin**.

CSS Box-sizing

- The box-sizing property controls how the width and height of an element are calculated

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 5px;  
}
```

Total width = 300 + 40 + 10 = 350px

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 5px;  
  box-sizing: border-box;  
}
```

Total width = content + padding + border = **defined width**

Total width = 300px;

Display Property

- block
 - Element takes the **full width** of its parent.
 - Starts on a new line.
 - Examples: <div>, <h1>, <p>.
- inline
 - Flows in the **same line** with text/content.
 - Width/height cannot be set.
 - Examples: , <a>, .
- inline-block
 - Behaves like **inline** (stays on the same line).
 - But **allows width and height settings**.
- none
 - Element is **not displayed** (removed from layout flow).
 - Different from visibility: hidden (hidden but space preserved).

Display Property - Example

```
<style>
```

```
.block {  
  display: block;  
  background: lightblue;  
  margin: 5px 0;  
}  
  
.inline {  
  display: inline;  
  background: lightgreen;  
  padding: 5px;  
  /* width/height ignored for inline */  
  width: 200px;  
  height: 50px;  
}
```

```
.wrapper {  
  border: 2px dashed black;  
  padding: 10px;  
  margin-top: 20px;  
}
```

```
</style>
```

```
<div class="wrapper">  
  <h2>Block</h2>  
  <div class="block">Block element 1</div>  
  <div class="block">Block element 2</div>  
</div>  
  
<div class="wrapper">  
  <h2>Inline</h2>  
  <p>  
    Some text  
    <span class="inline">Inline span</span>  
    continues on the same line  
    <a href="#" class="inline">Inline link</a>.  
  </p>  
</div>
```

Block

Block element 1

Block element 2

Inline

Some text Inline span continues on the same line [Inline link](#).

Position Property

- static (default): is not affected by top, bottom, left, right.
- relative
 - Positioned relative to its normal position.
 - Can use top, left, etc. to shift slightly.
- absolute
 - Positioned relative to the nearest positioned ancestor
 - If no ancestor, positioned relative to the document <html>.
- fixed: stays in place when scrolling.
- sticky: is positioned based on the user's scroll position.

```
div.relative {  
  position: relative;  
  left: 20px;  
  width: 400px;  
  height: 200px;  
  border: 3px solid #73AD21;  
}
```

```
div.absolute {  
  position: absolute;  
  top: 80px;  
  right: 0;  
  width: 200px;  
  height: 100px;  
  border: 3px solid #73AD21;  
}
```

This div element has position: relative;

This div element has position:
absolute;

Position

```
.static {  
  position: static;  
  width: 300px;  
  height: 100px;  
  left: 50px;  
  border: 3px solid red;  
}
```

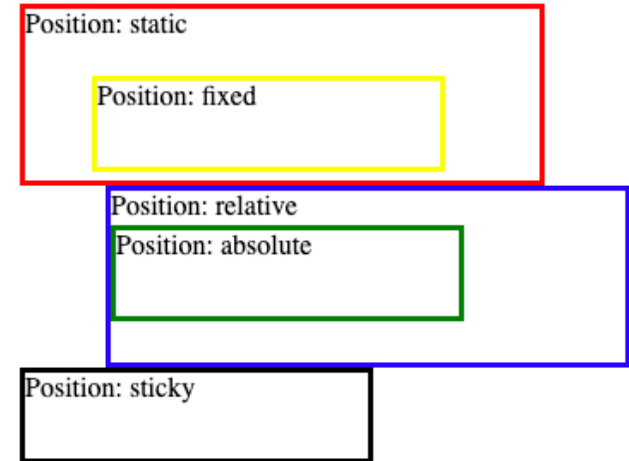
```
.relative {  
  position: relative;  
  width: 300px;  
  height: 100px;  
  left: 50px;  
  border: 3px solid blue;  
}
```

```
.absolute {  
  position: absolute;  
  top: 20px;  
  width: 200px;  
  height: 50px;  
  border: 3px solid green;  
}
```

```
.fixed {  
  position: fixed;  
  top: 50px;  
  left: 50px;  
  width: 200px;  
  height: 50px;  
  border: 3px solid yellow;  
}
```

```
.sticky {  
  position: sticky;  
  top: 20px;  
  width: 200px;  
  height: 50px;  
  border: 3px solid black;  
}
```

```
</style>  
</head>  
<body>  
  <div class="static">Position: static</div>  
  <div class="relative">  
    Position: relative  
    <div class="absolute">Position: absolute</div>  
  </div>  
  <div class="fixed">Position: fixed</div>  
  <div class="sticky">Position: sticky</div>  
</body>
```



Overlap

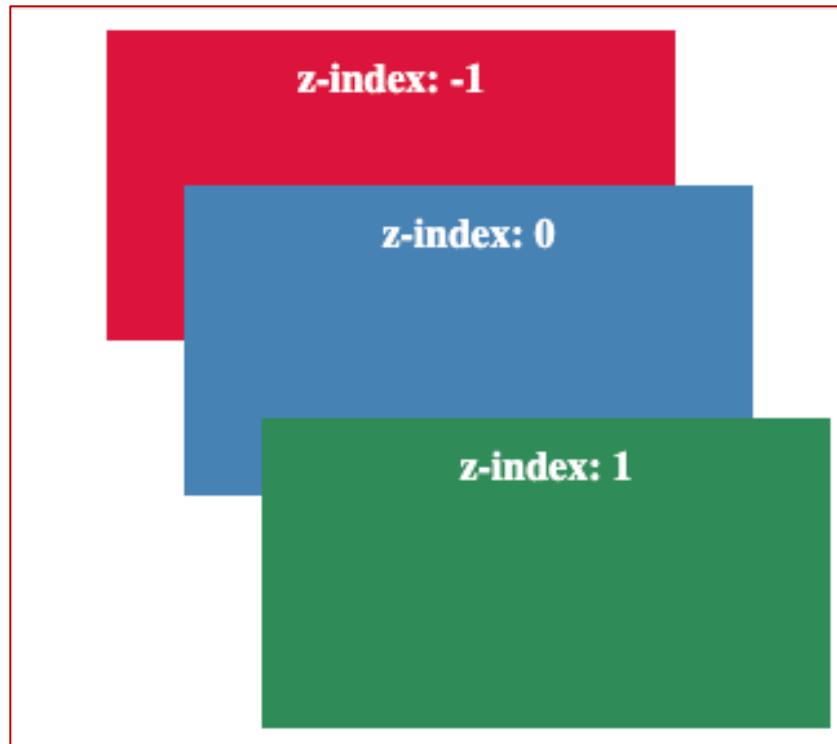
```
<style>
.box {
  position: relative;
  width: 200px;
  height: 100px;
  padding: 10px;
  color: □white;
  font-weight: bold;
  text-align: center;
}

.z-neg {
  background: ■crimson;
  left: 30px;
  z-index: -1; /* nằm dưới cùng */
}

.z-zero {
  background: ■steelblue;
  top: -60px;
  left: 60px;
  z-index: 0; /* mặc định */
}

.z-pos {
  background: ■seagreen;
  top: -90px;
  left: 90px;
  z-index: 1; /* trên cùng */
}
</style>
```

```
<body>
  <div class="box z-neg">z-index: -1</div>
  <div class="box z-zero">z-index: 0</div>
  <div class="box z-pos">z-index: 1</div>
</body>
```



1.5. CSS Variables

- CSS variables store reusable values
- Defined using `--variable-name`
- Accessed using `var()`

```
:root{  
  --main-color: blue;  
  --padding-size: 16px;  
}  
h1{  
  color: var(--main-color);  
}  
.box{  
  padding: var(--padding-size);  
}
```



2. Responsive Web Design

2.1. Concepts & Principles

2.2. Media Query

2.3. Flexbox

2.4. Grid



2.1. Concepts & Principles

- Design for multi-devices with different screen sizes (mobile, tablet, desktop).
- The layout adapts automatically to the device being used.



Example

Column 1

Column 2

Column 3

Mobile

Column 1

Column 2

Column 3

Tablet

Column 1

Column 2

Column 3

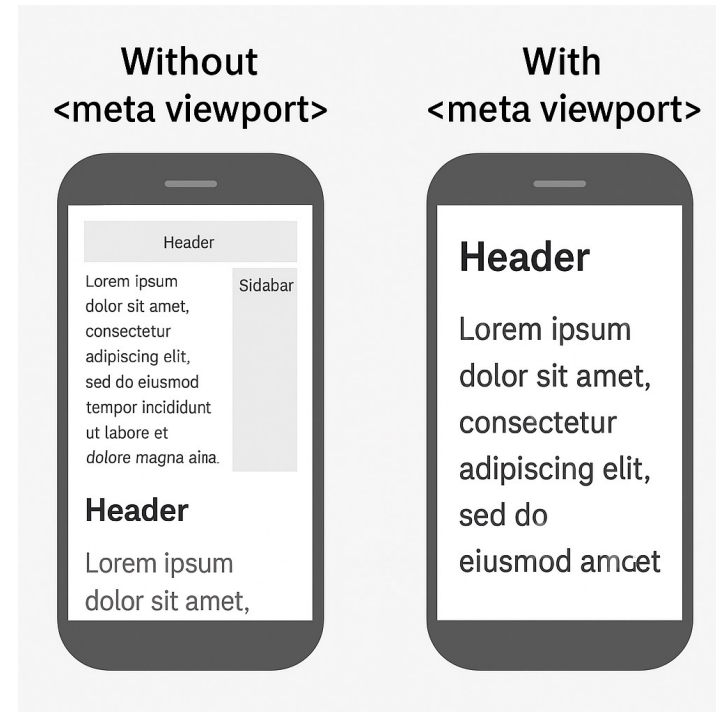
Laptop/Desktop

Meta Viewport

- The `<meta viewport>` tag tells the browser how to control the page's dimensions and scaling.
- Essential for **responsive design** on mobile and tablet devices.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

- **viewport** → visible area of a web page (mobile, table, desktop)
- **width=device-width** → Page width matches the device screen width.
- **initial-scale=1.0** → Sets the default zoom level when the page loads.



Flexible Image

- Images should adapt to the size of their container.
- Use max-width: 100% to make images scale down when needed.
- Prevents images from overflowing outside their parent element.

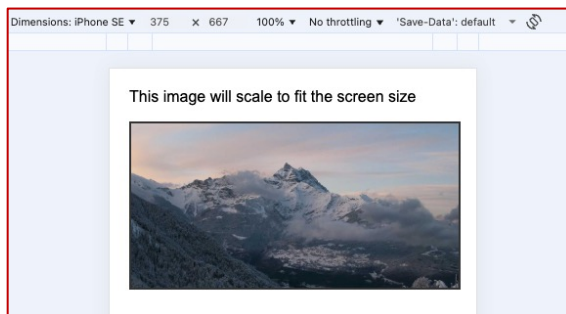
```
img {  
    max-width: 100%;  
    height: auto;  
}
```

- max-width: 100% → image never exceeds container width.
- height: auto → keeps aspect ratio when scaling.
- Result: image automatically resizes on smaller screens

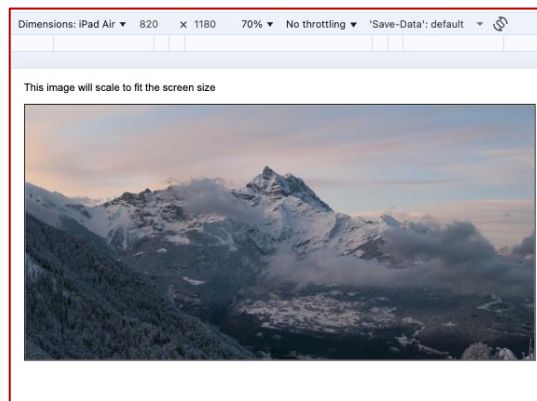
```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Image Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    img {
      max-width: 100%;
      height: auto;
      border: 2px solid #333;
      display: block;
      margin: auto;
    }
  </style>
</head>
<body>
  <p>This image will scale to fit the screen size</p>
  
</body>

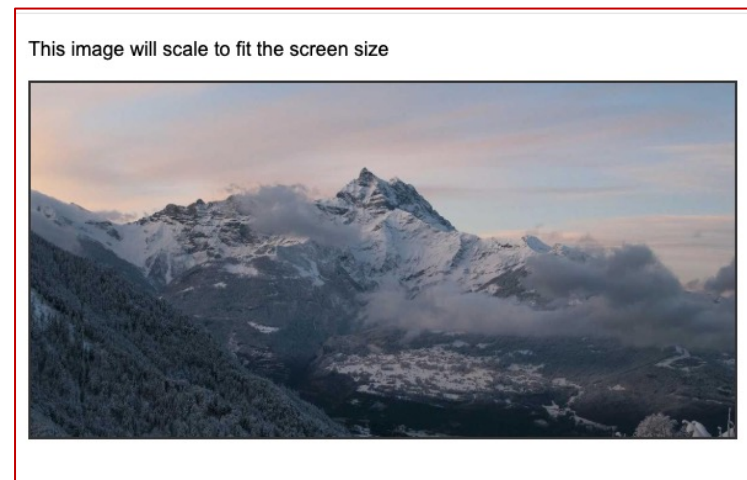
```



Mobile



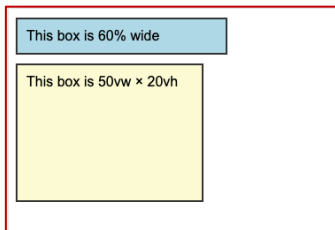
Tablet



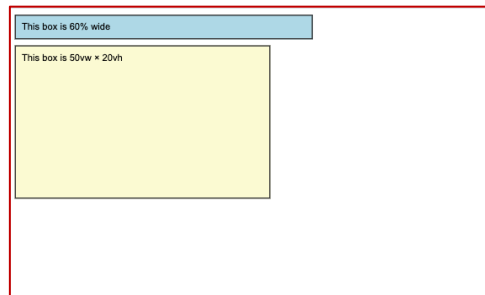
Laptop/Desktop 46

Relative Units

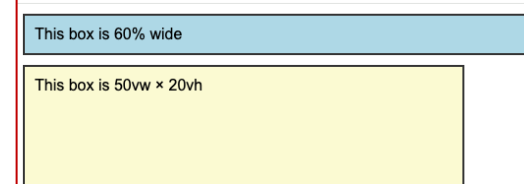
```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Relative Units Simple Demo</title>
  <style>
    body { font-family: Arial, sans-serif; }
    .box {
      border: 2px solid ■ #333;
      margin: 10px 0;
      padding: 10px;
    }
    .percent { width: 60%; background: ■ lightblue; }
    .vwvh { width: 50vw; height: 20vh; background: ■ lightgoldenrodyellow; }
  </style>
</head>
<body>
  <div class="box percent">This box is 60% wide</div>
  <div class="box vwvh">This box is 50vw x 20vh</div>
</body>
```



Mobile



Tablet



Laptop/Desktop

2.2. Media Query

- Allow applying CSS conditionally based on device characteristics.
- Width/Height of viewport (max-width, min-width), e.g.,
 - 0px - 576px: mobile
 - 577px - 992px : tablet
 - 993px+ : laptop, desktop
- Media type (screen, print, all)

```
@media (max-width: 576px)
{
  body {
    font-size: 14px;
  }
}
```

```
@media print {
  body {
    font-size: 12pt;
    color: black;
    background: none; /*bỏ nền*/
  }
}
```

Media Query - Example

```
<style>
  /* Default */
  body {
    font-family: Arial;
    background: lightblue;
  }

  /* Print */
  @media print {
    body { background: none; color: black;}
  }
</style>
```

Print this page to see different effects.

Print this page to see different effects.

2.3. Flexbox

- **Flexbox** is a one-dimensional layout system (rows or columns).
- Works well for **responsive layouts** because elements can wrap and align easily.
- Main properties
 - **flex-wrap: wrap;** → Items automatically move to a new line if space is not enough.
 - **justify-content** → Align items horizontally
 - **align-items** → Align items vertically

2.3. Flexbox

- **flex: flex-grow flex-shrink flex-basis;**
 - flex-grow → how much the item will grow relative to others
 - flex-shrink → how much the item will shrink relative to others
 - flex-basis → the initial size of the item

```
.item1 { flex: 1 1 200px; }  
/* flex-grow = 1, flex-shrink = 1, flex-basis = 200px */  
  
.item2 { flex: 2; }  
/* shorthand for: flex-grow = 2, flex-shrink = 1, flex-basis = 0 */
```

- `flex: 1 1 200px;`
 - The item starts with 200px width, can grow to fill extra space, and can shrink if needed.
- `flex: 2;`
 - The item will grow at twice the rate of items with `flex: 1`.

Example

```
<style>
  .container {
    display: flex;
    flex-wrap: wrap;          /* Cho phép xuống dòng khi thiếu chỗ */
    justify-content: space-between; /* Căn đều ngang, có khoảng cách */
  }
  .box {
    flex: 1 1 200px; /* Grow, shrink, base width */
    margin: 10px;
    background-color: lightblue;
    text-align: center;
    padding: 20px;
    font-size: 18px;
  }
</style>
```

</head>

<body>

```
<div class="container">
```

```
  <div class="box">Box 1</div>
```

```
  <div class="box">Box 2</div>
```

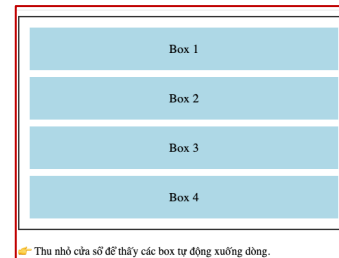
```
  <div class="box">Box 3</div>
```

```
  <div class="box">Box 4</div>
```

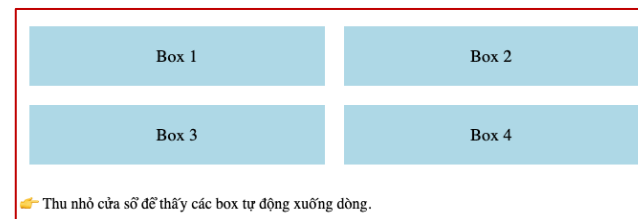
```
</div>
```

```
<p>☞ Thu nhỏ cửa sổ để thấy các box tự động xuống dòng.</p>
```

</body>



☞ Thu nhỏ cửa sổ để thấy các box tự động xuống dòng.



☞ Thu nhỏ cửa sổ để thấy các box tự động xuống dòng.

2.4. Grid

- CSS Grid is a **two-dimensional layout system**
- It organizes elements into **rows and columns**
- Suitable for **complex page layouts**
- Compared with Flexbox (1D layout)

```
<style>
.container {
  display: grid;
  grid-template-columns: auto auto auto;
  background-color: dodgerblue;
  padding: 10px;
}
.container div {
  background-color: #f1f1f1;
  border: 1px solid black;
  padding: 10px;
  font-size: 30px;
  text-align: center;
}
```

```
<body>
<div class="container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
</div>
</body>
```



Basic Grid Syntax

- Basic Grid Syntax

`display: grid;`

→ grid layout

- Define Columns

`grid-template-columns:
1fr 1fr 1fr;`

→ 3 columns of equal width
(each column = 1 fraction unit)

- Add Space Between Items

`gap: 20px;`

```
.container {  
  display: grid;  
  grid-template-columns:  
1fr 1fr 1fr;  
  gap: 20px;  
}
```

```
+-----+-----+-----+  
| 1 | 2 | 3 |  
+-----+-----+-----+  
| 4 | 5 | 6 |  
+-----+-----+-----+
```

Grid vs Flexbox

Feature	Flexbox	Grid
Layout type	1-dimensional	2-dimensional
Direction	Row OR Column	Row AND Column
Best use	Navigation bar	Page layout

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in a way that creates a sense of depth and movement, resembling a spiral or a stylized 'H' shape. The word 'HUST' is written in white, bold, sans-serif capital letters, centered within the blue area.

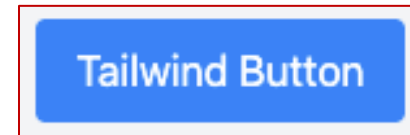
HUST

3. CSS Framework

What is Tailwind CSS?

- A utility-first CSS framework
- Focuses on building UI directly in HTML without writing new CSS files.
- Benefits
 - Fast development: uses predefined utility classes
 - Consistency: avoid messy custom CSS
 - Maintainability: easy to read and modify

```
<button class="bg-blue-500 text-white py-2 px-4 rounded">  
  Tailwind Button  
</button>
```



- background color: Tailwind's blue at level 500
- text: white color
- py-2: vertical padding (top, bottom) to 0.5rem
- px-4: horizontal paddign (left, right) to 1 rem

How to install Tailwind CSS

1. Using CDN (Quick Demo)

- Easiest way to start without setup.
- Add this line inside <head>:

```
<script src="https://cdn.tailwindcss.com"></script>
```

2. Other ways

<https://tailwindcss.com/docs/installation/tailwind-cli>

Typography

- Text Size
 - text-lg → Large text ($\approx 18\text{px}$).
 - text-2xl → Extra large text ($\approx 24\text{px}$).
- Font Weight
 - font-bold → Bold text.
- Text Alignment
 - text-center → Centers the text horizontally.
 - Other options: text-left, text-right, text-justify
- Text Style
 - italic → Italic text.

Typography Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tailwind Typography Simple</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="p-6 font-sans">

  <p class="text-lg">Large text (text-lg)</p>
  <p class="text-2xl font-bold">Extra large & bold (text-2xl font-bold)</p>
  <p class="text-center">This text is centered (text-center)</p>
  <p class="italic">Italic text (italic)</p>
  <p class="underline">Underlined text (underline)</p>

</body>
</html>
```

Large text (text-lg)

Extra large & bold (text-2xl font-bold)

This text is centered (text-center)

Italic text (italic)

Underlined text (underline)

Color

- text-red-500 → Red text.
- bg-blue-200 → Light blue background.
- bg-gradient-to-r → Gradient background from left to right.

```
<body class="space-y-3">  
  
  <p class="text-red-500">This is red text</p>  
  <p class="bg-blue-200 p-2">This has a light blue background</p>  
  <div class="bg-gradient-to-r from-green-400 to-blue-500 text-white p-2">  
    Gradient background  
  </div>  
  
</body>
```

This is red text

This has a light blue background

Gradient background

- **Margin (m) Utilities**

- m-4 → Applies margin on **all sides** (1rem ≈ 16px).
- mt-2 → Margin on **top only** (0.5rem ≈ 8px).
- mx-auto → Horizontal margin set to **auto**, often used for centering block elements.

- **Padding (p) Utilities**

- p-4 → Padding on **all sides** (1rem ≈ 16px).
- px-6 → Horizontal padding (left & right = 1.5rem).
- py-2 → Vertical padding (top & bottom = 0.5rem).

Spacing Example

```
<body class="p-6 bg-gray-100 font-sans">

  <!-- Margin Example -->
  <div class="bg-blue-200 m-4">
    <p>Box with <code>m-4</code> (margin around the box)</p>
  </div>

  <!-- Padding Example -->
  <div class="bg-green-200 p-6 mt-6">
    <p>Box with <code>p-6</code> (padding inside the box)</p>
  </div>

  <!-- Combined Example -->
  <div class="bg-yellow-200 m-4 p-4 mt-6">
    <p>Box with <code>m-4 p-4</code> (both margin and padding)</p>
  </div>

</body>
```

Box with m-4 (margin around the box)

Box with p-6 (padding inside the box)

Box with m-4 p-4 (both margin and padding)

Responsive in Tailwind CSS

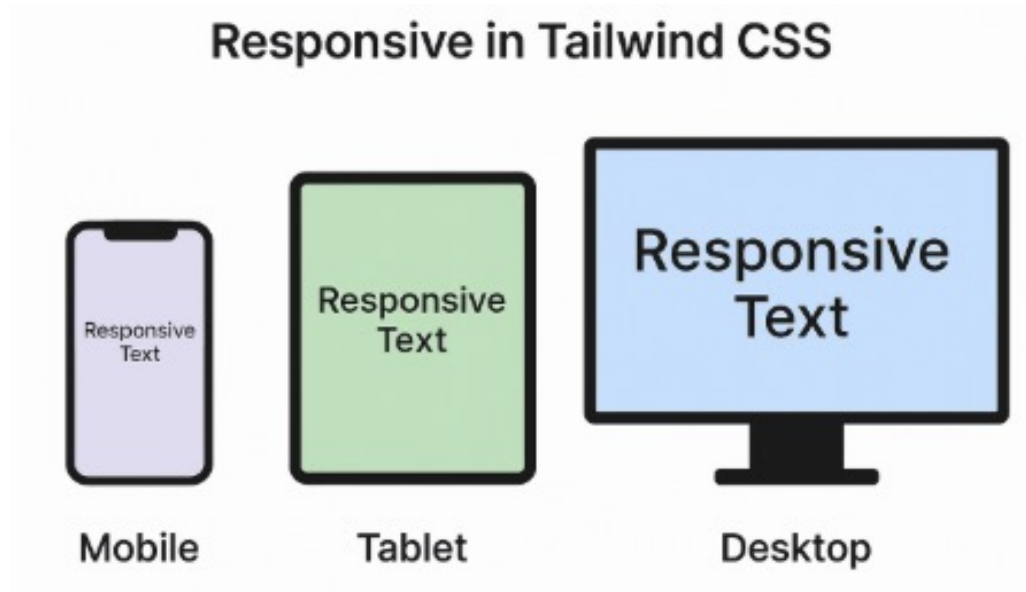
- Tailwind provides responsive utility classes using prefixes.
- Prefixes apply styles only when the screen width is greater than or equal to a certain breakpoint.
- Common Breakpoints
 - sm: $\rightarrow \geq 640\text{px}$ (small devices, tablets in portrait)
 - md: $\rightarrow \geq 768\text{px}$ (medium devices, tablets in landscape)
 - lg: $\rightarrow \geq 1024\text{px}$ (large devices, laptops)

Responsive in Tailwind CSS - Example

```
<!-- Responsive Box -->
```

```
<div class="w-32 h-32 bg-purple-400 sm:bg-green-400 lg:bg-blue-400">
```

```
</div>
```



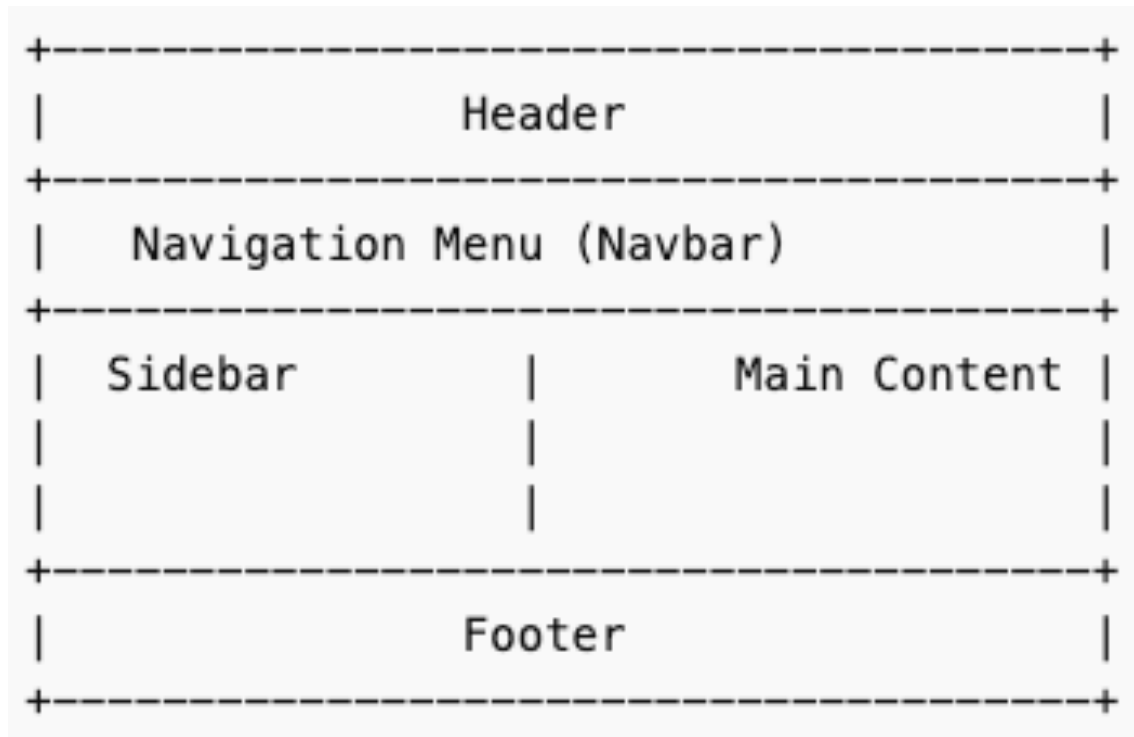
A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly irregular rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

4. Webpage Layout

Typical Web Page Layout

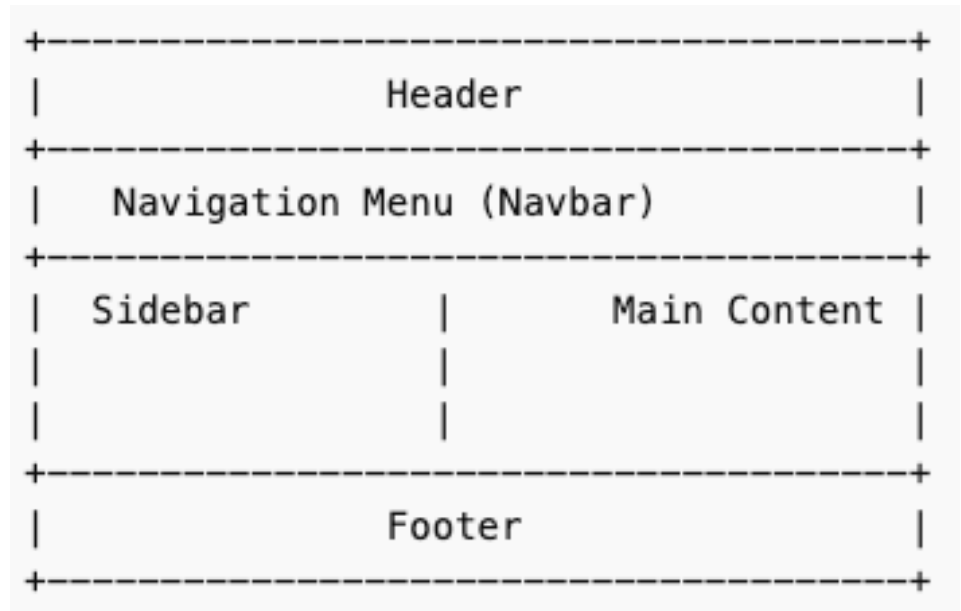
- Most websites are organized into several common sections
- Purpose: improve readability and navigation



Layout Using CSS Flexbox

- Flexbox can be used to arrange elements in a row or column

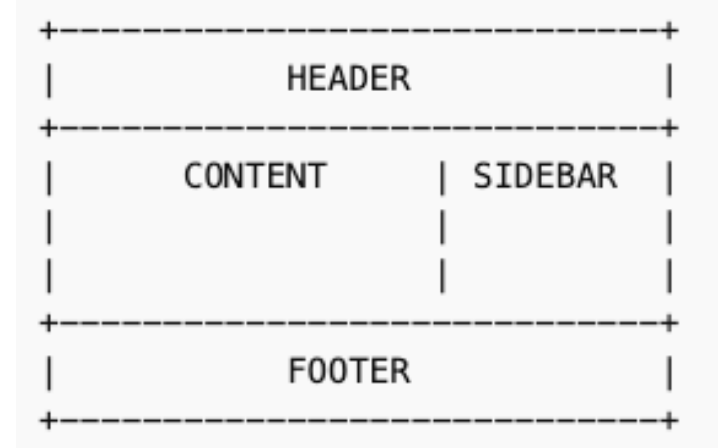
```
.container {  
  display: flex;  
}  
  
.sidebar {  
  width: 25%;  
}  
  
.main {  
  width: 75%;  
}
```



Layout using CSS Grid

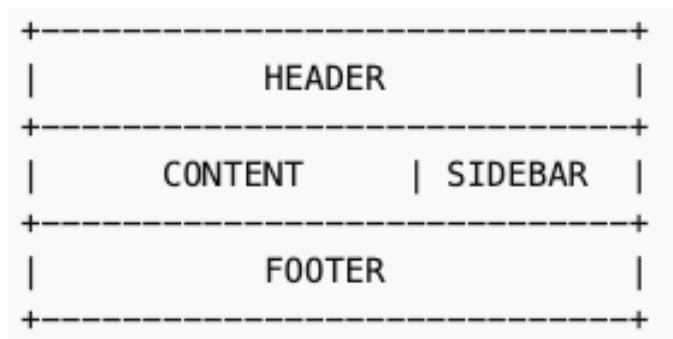
- Modern websites often use **Grid Layout**

```
.container{  
  display:grid;  
  grid-template-columns: 3fr 1fr;  
  grid-template-rows:auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "content sidebar"  
    "footer footer";  
}
```

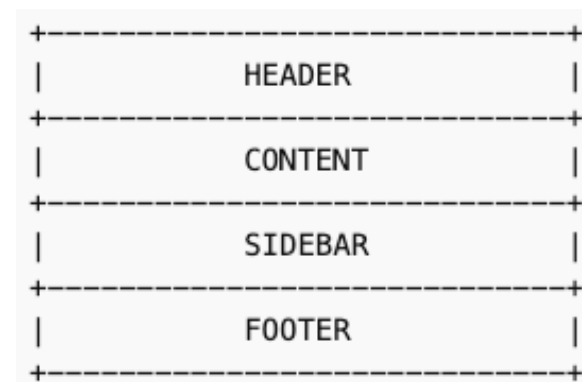


Responsive Layout

```
@media (max-width: 768px) {  
  .container {  
    grid-template-columns: 1fr;  
    grid-template-areas:  
      "header"  
      "content"  
      "sidebar"  
      "footer";  
  }}
```



Desktop



Mobile

Exercises

- Ex1. Create a simple page based on HTML and CSS
- Ex2. Create a page following responsive web design

Exercises

- <https://www.freecodecamp.org/learn/2022/responsive-web-design/>
- Exercises 3, 4:
 - Learn HTML by Building a Cat Photo App
 - Build a Personal Portfolio Webpage

